Sistemas Operacionais

Sistema de arquivos

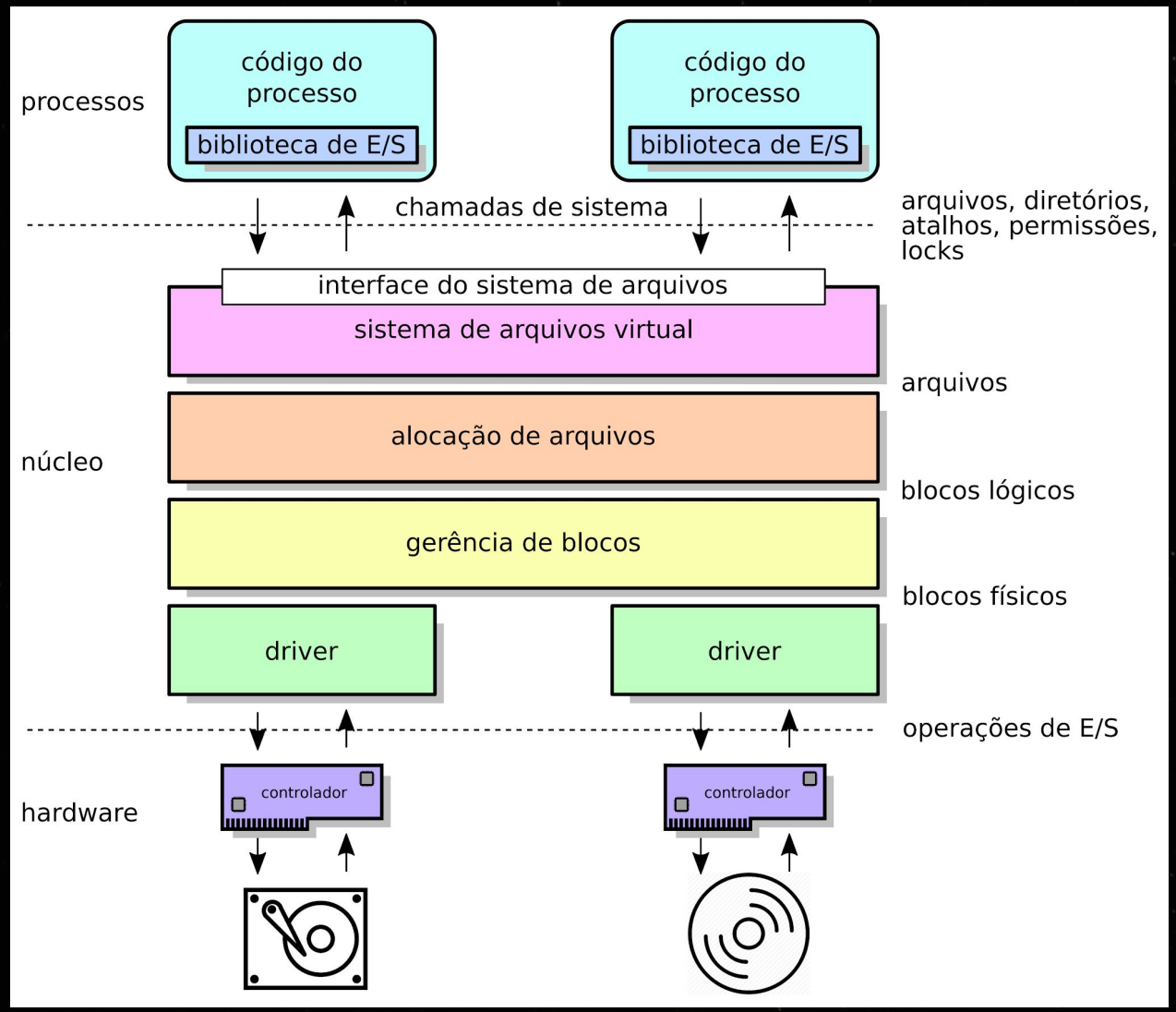
Professor Romário da Silva Borges

Sistemas de arquivos

Subsistemas do sistema operacional que implementam o conceito de arquivo sobre o armazenamento bruto proporcionado pelos dispositivos físicos.

Organizados em camadas: do hardware às bibliotecas de E/S usadas pelos processos.

Arquitetura em camadas



Dados e metadados

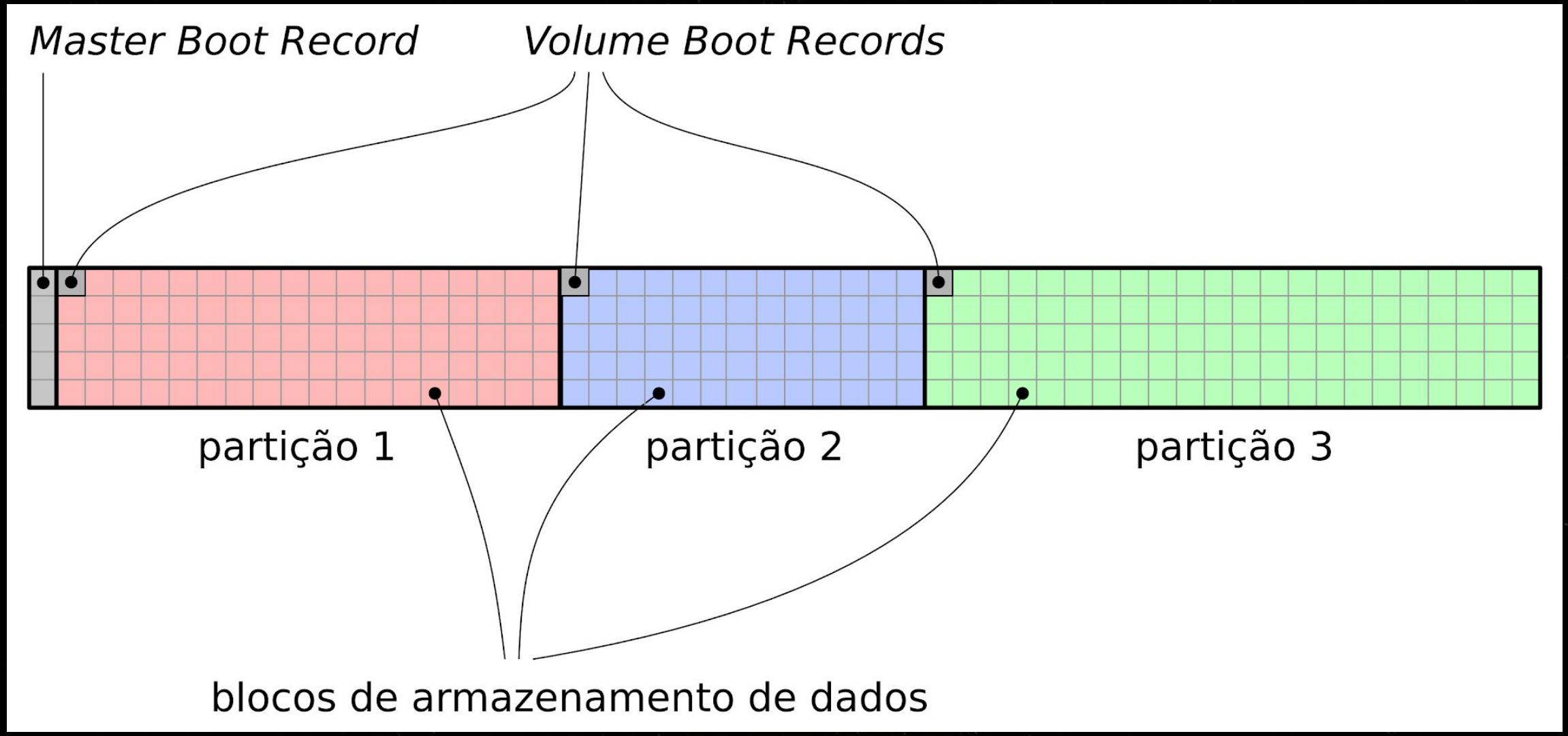
- **Dados:** o conteúdo do arquivo em si (uma música, uma foto, um documento).
- **Metadados:** atributos (nome, datas, permissões) e informações para localizar o conteúdo no disco.
- Mapas de blocos livres também são metadados.

Espaços de armazenamento

Discos e partições

- O disco é visto como **um grande vetor de blocos** de tamanho fixo.
- **MBR:** tabela de partições e código de boot do disco.
- **VBR:** bloco de inicialização de cada partição.
- Cada volume é **formatado** com seu próprio sistema de arquivos.

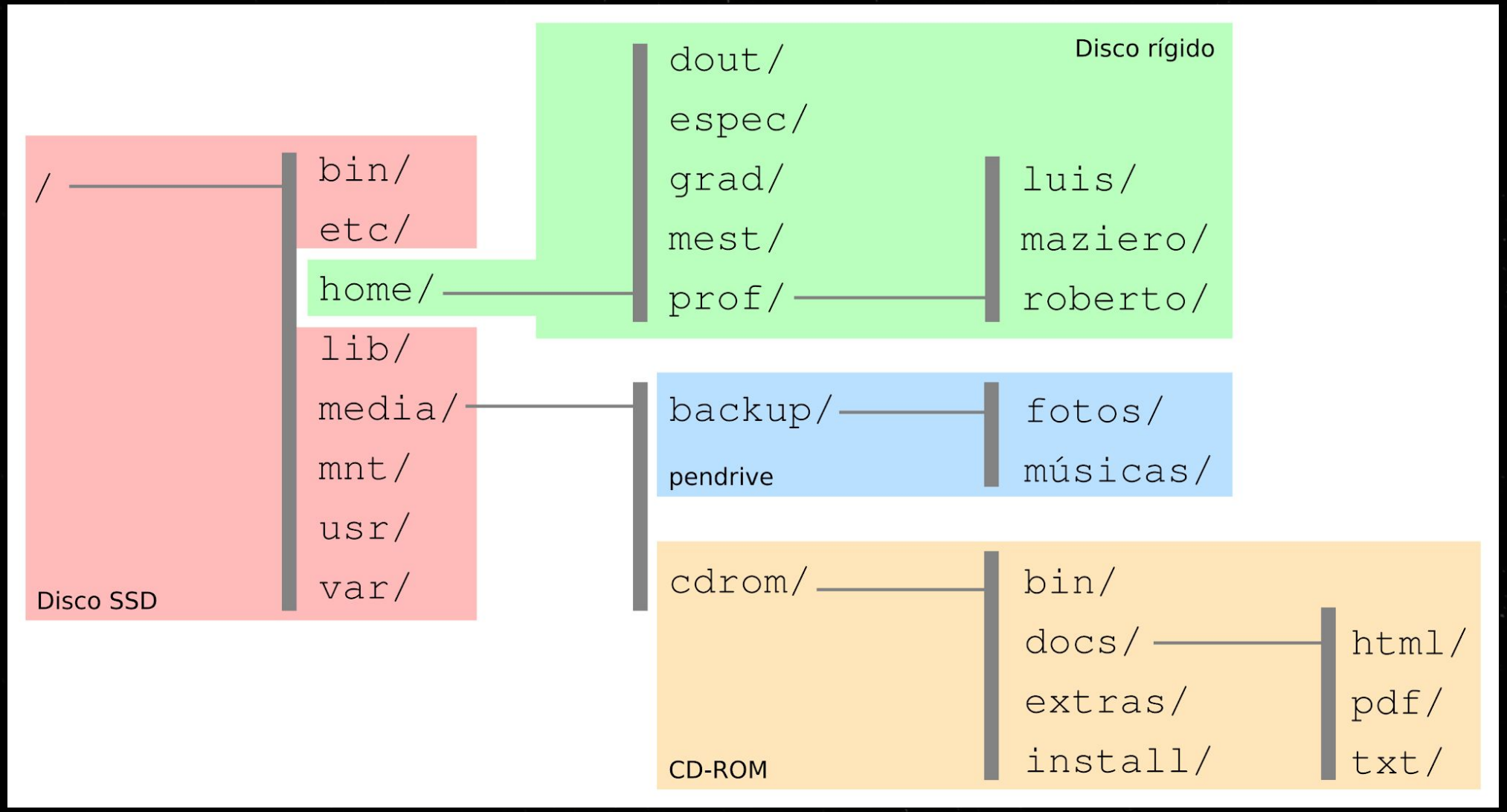
Organização em partições



Montagem de volumes

- **Montar:** ler o bloco de inicialização, criar as estruturas em memória e definir um ponto de acesso.
- **UNIX:** volumes viram subdiretórios da árvore (mount).
- **Windows:** cada volume vira um disco lógico (C:, D:).

Montagem de volumes no UNIX



Gestão de blocos

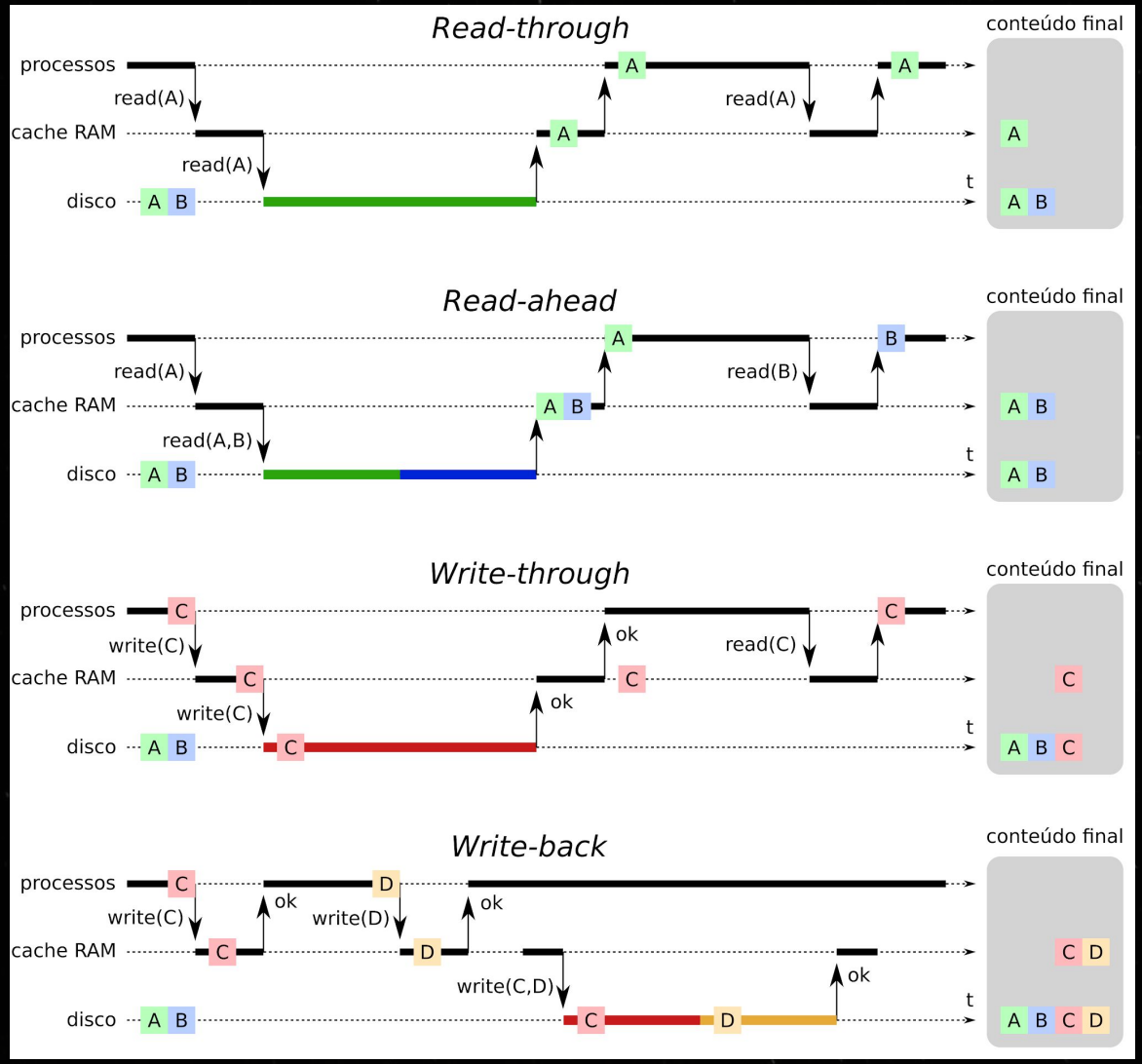
Blocos físicos e lógicos

- Blocos físicos (512 B ou 4 KB) são agrupados em **blocos lógicos** (clusters).
- O bloco lógico é a unidade mínima de alocação.
- Blocos grandes: mais eficiência de E/S e mais **fragmentação interna**.
- Ex.: um arquivo de 200 bytes ocupa um bloco lógico inteiro.

Caching de blocos

- **Read-through** e **read-ahead** aceleram as leituras.
- **Write-through:** escrita síncrona, sem ganho na escrita.
- **Write-back:** libera o processo antes, mas pode perder dados num crash.
- Dados usam write-back; metadados usam write-through.

Estratégias de caching

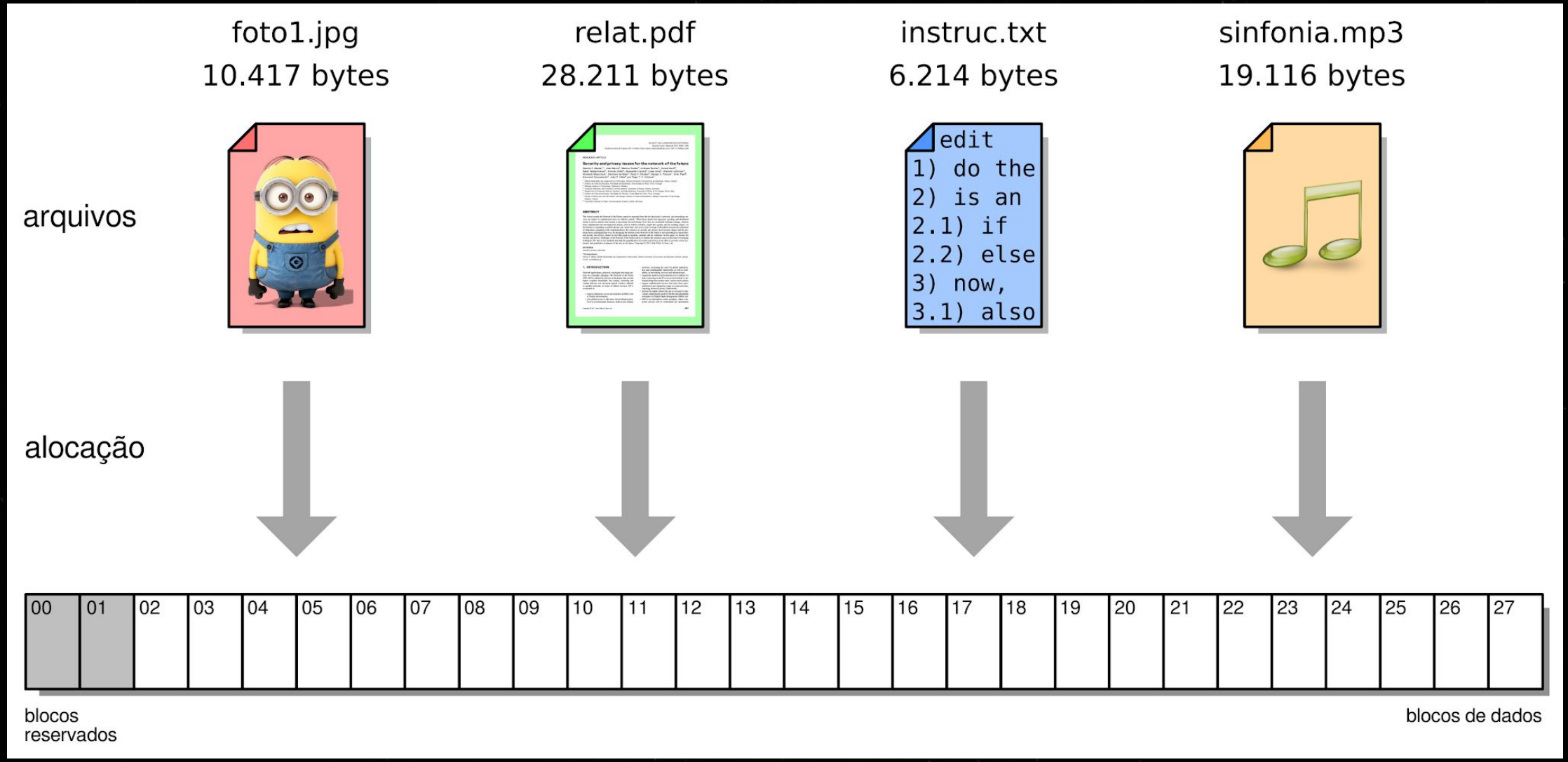


Alocação de arquivos

O problema da alocação

- Dispor conteúdo e metadados dos arquivos nos blocos lógicos do disco.
- Critérios: **rapidez, robustez e flexibilidade**.
- **FCB** (File Control Block): metadados e localização do conteúdo.

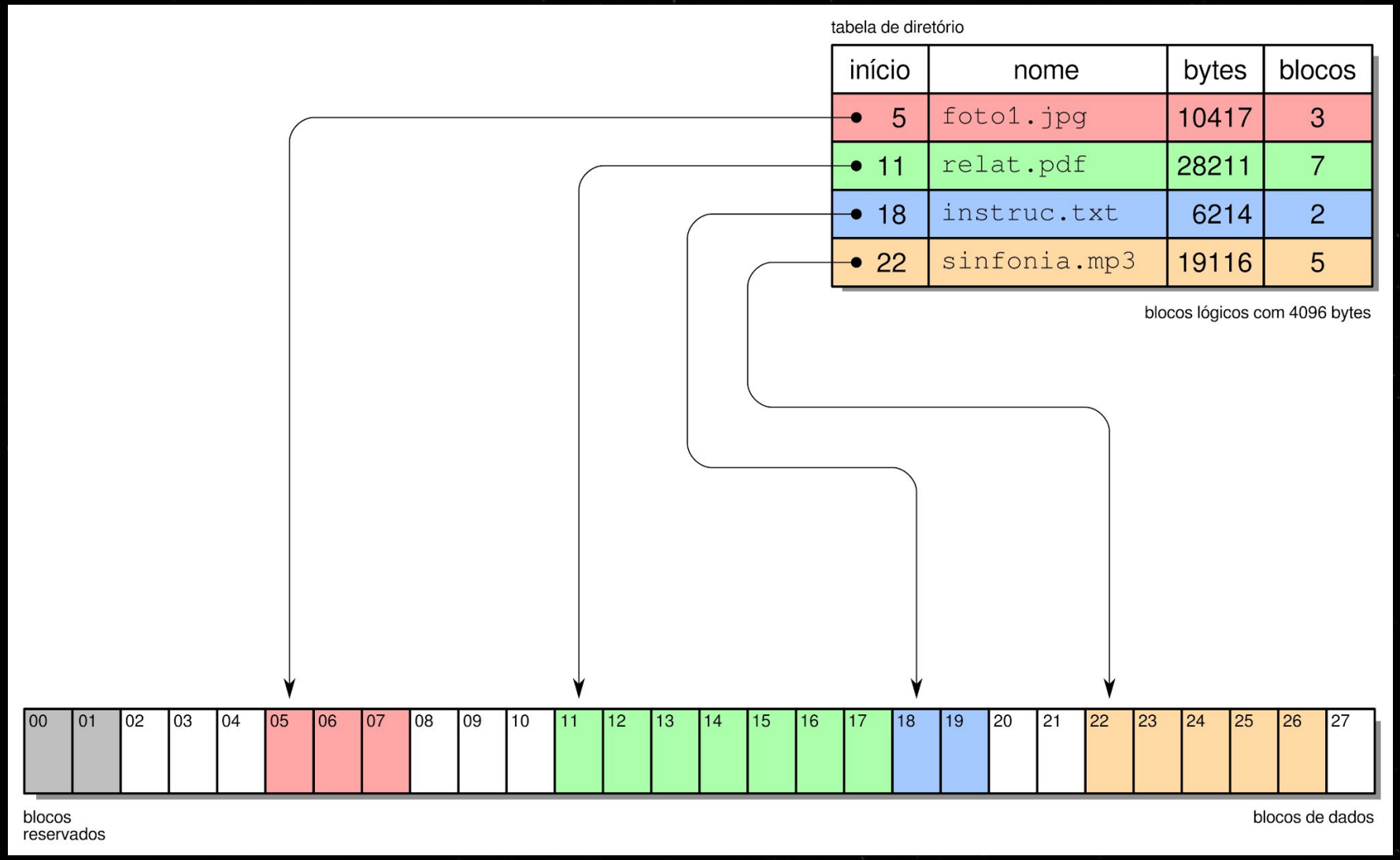
O problema da alocação



Alocação contígua

- Blocos consecutivos: basta o **endereço do primeiro bloco**.
- Acesso sequencial e aleatório rápidos; boa robustez.
- Baixa flexibilidade e **fragmentação externa**.
- Ex.: ISO 9660, usado em CD-ROMs.

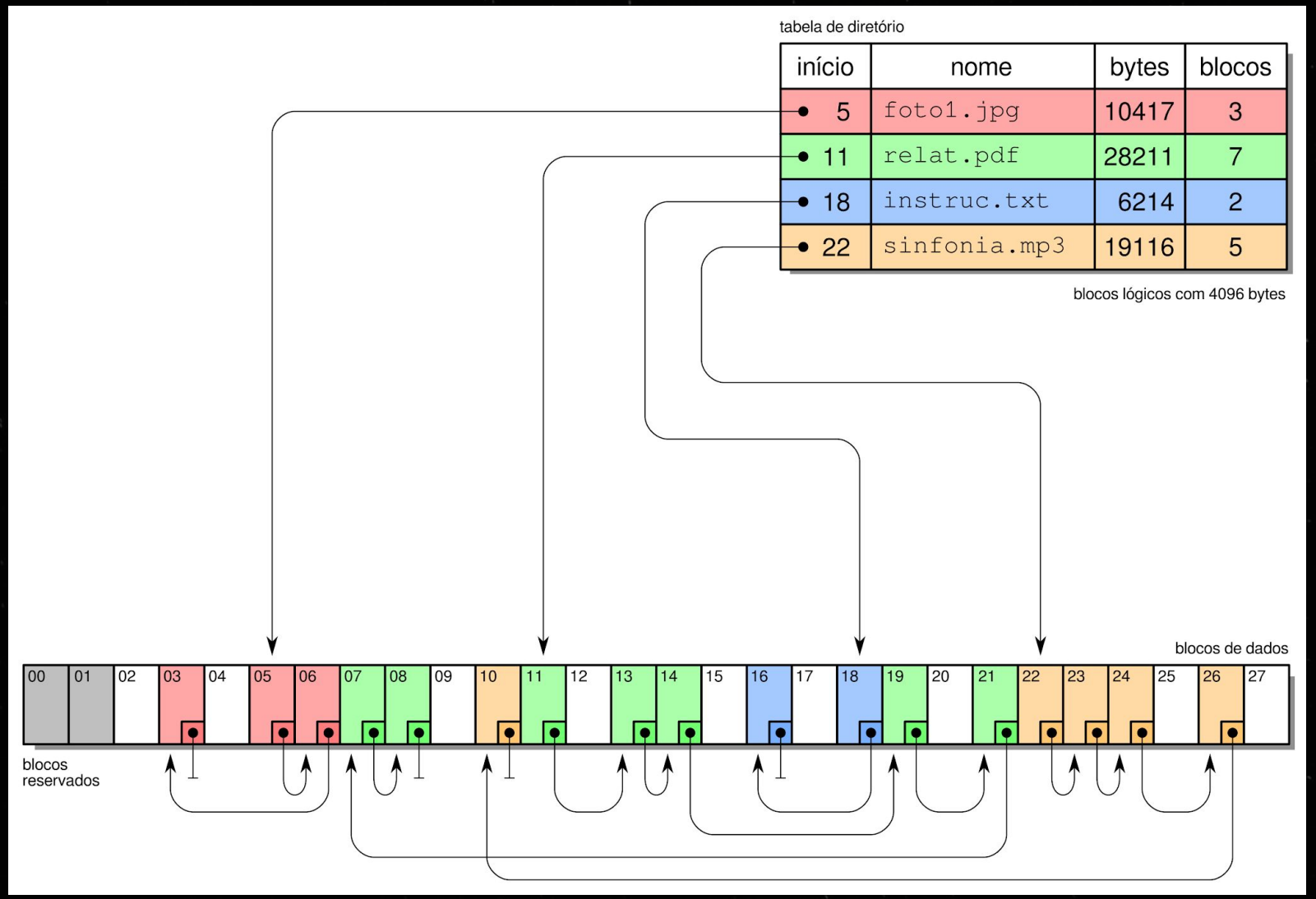
Alocação contígua



Alocação encadeada

- Cada bloco guarda dados e **um ponteiro para o próximo**.
- Sem fragmentação externa; o arquivo cresce livremente.
- Acesso aleatório lento: é preciso ler os $n - 1$ blocos anteriores.
- Um bloco defeituoso torna inacessíveis os seguintes.

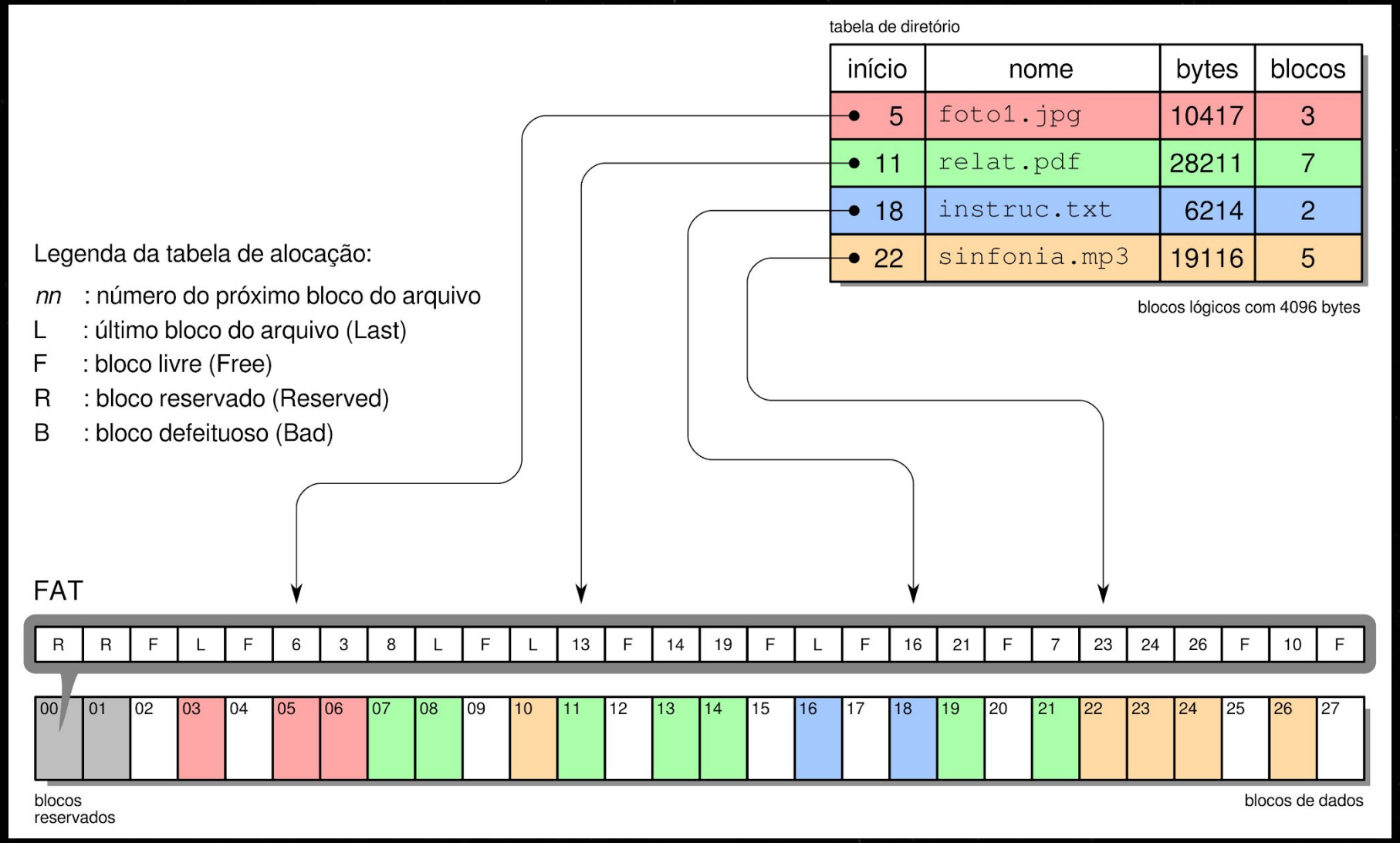
Alocação encadeada simples



Alocação encadeada FAT

- Os ponteiros saem dos blocos e vão para uma tabela: a **FAT**.
- Mantém a flexibilidade e acelera o acesso aleatório.
- A FAT é um dado crítico: cópias ficam na área reservada.
- Usada em FAT12/16/32, pendrives e câmeras.

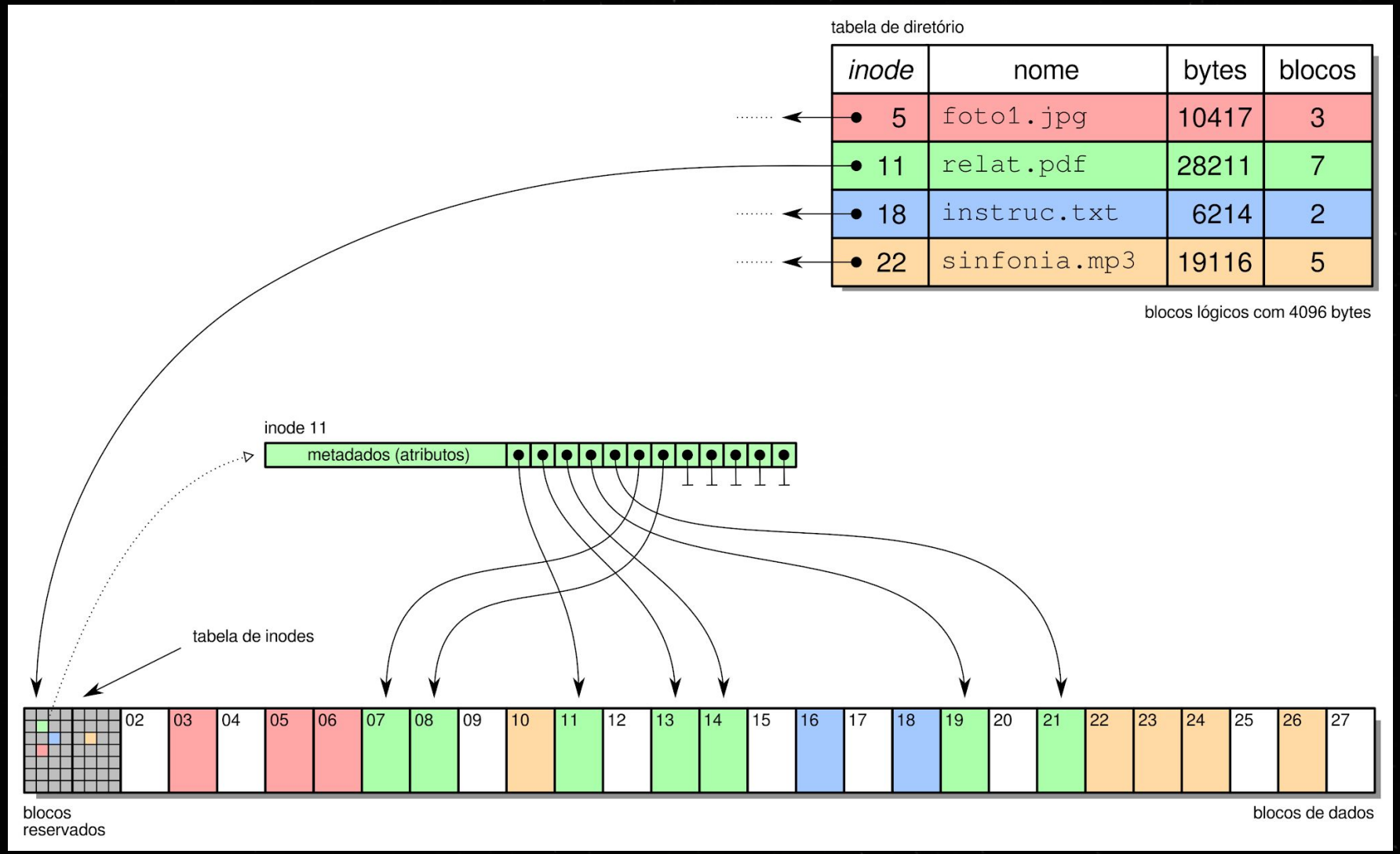
Alocação encadeada com FAT



Alocação indexada

- Cada arquivo tem um **i-node**: atributos e índice de blocos.
- Acesso sequencial e aleatório rápidos.
- O i-node tem tamanho fixo e limita o tamanho do arquivo.
- A tabela de i-nodes limita o número de arquivos.

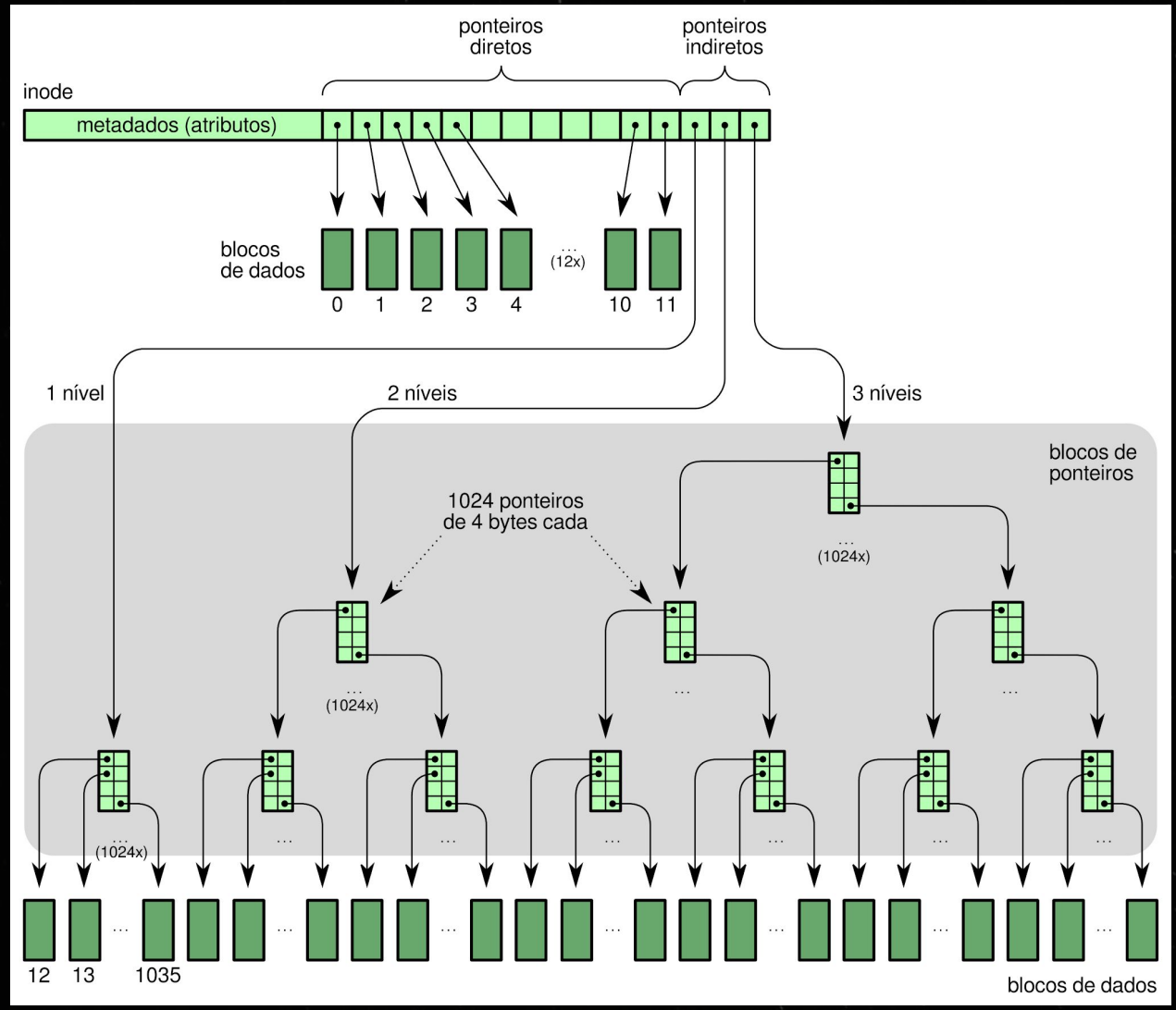
Alocação indexada simples



Alocação indexada multinível

- **Ponteiros indiretos** apontam para blocos com mais ponteiros.
- Ext2/3/4: 12 diretos, 1 indireto, 1 duplo e 1 triplo.
- Com blocos de 4 KB: arquivos de até ≈ **4 TB**.
- Blocos no fim de arquivos grandes exigem até 3 leituras extras.

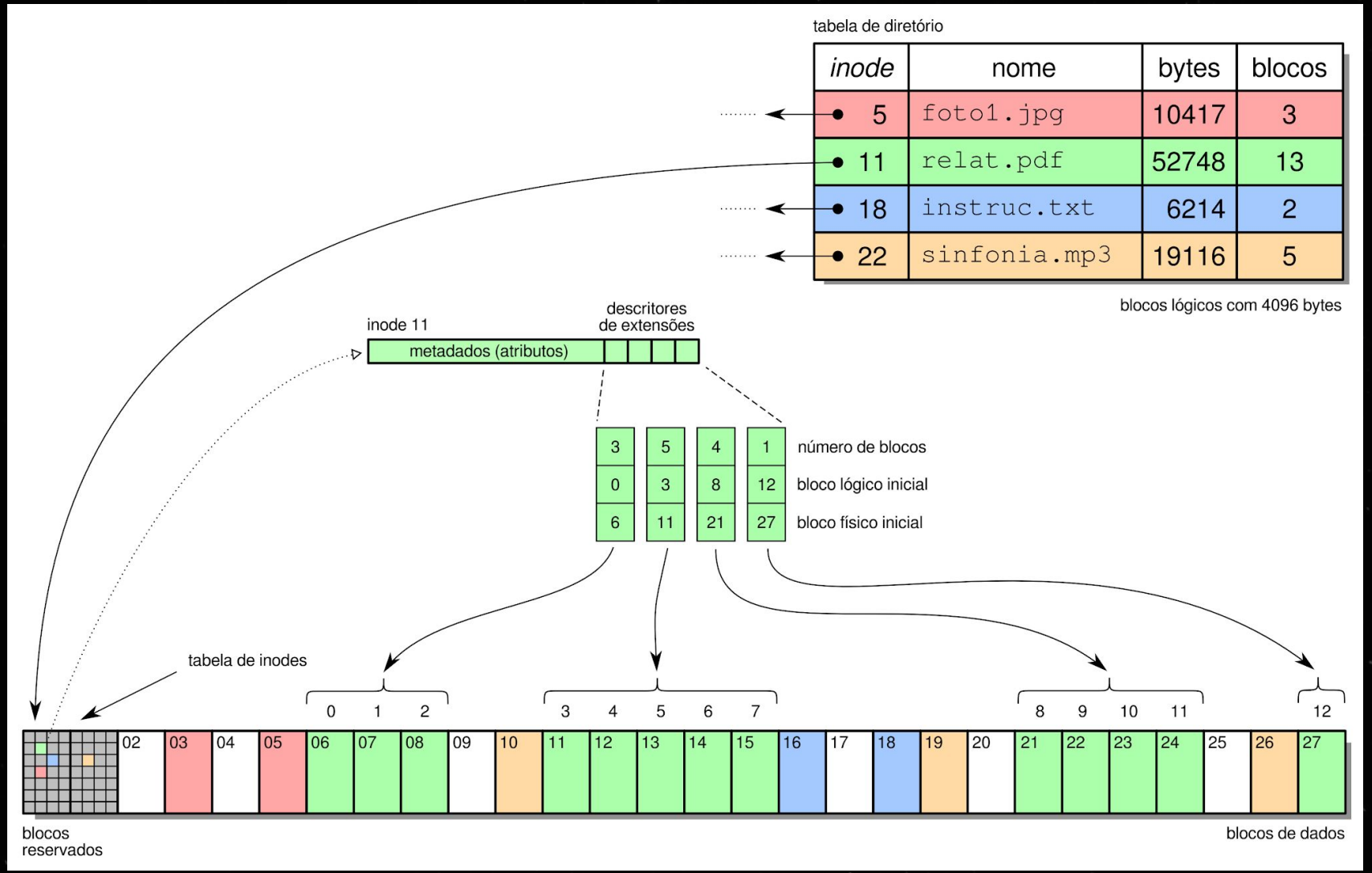
Alocação indexada multinível



Alocação por extensões

- **Extensão:** grupo de blocos contíguos (extent).
- Por extensão: bloco lógico inicial, bloco físico inicial e quantidade.
- Combina a economia da contígua com a flexibilidade da indexada.
- Usada em NTFS, HFS, Ext4 e ZFS.

Alocação por extensões



Análise comparativa

Estratégia	Contígua	Encadeada	FAT	Indexada	Extensões
Rapidez (acesso sequencial)	Alta, os blocos do arquivo estão sempre em sequência no disco.	Alta, se os blocos do arquivo estiverem próximos no disco.	Alta, se os blocos do arquivo estiverem próximos no disco.	Alta, se os blocos do arquivo estiverem próximos no disco.	Alta, se houver poucas extensões.
Rapidez (acesso aleatório)	Alta, as posições dos blocos podem ser calculadas sem acessar o disco.	Baixa, é necessário ler todos os blocos a partir do início do arquivo até encontrar o bloco desejado.	Alta, se os blocos do arquivo estiverem próximos no disco.	Alta, se os blocos do arquivo estiverem próximos no disco.	Alta, se houver poucas extensões.
Robustez	Alta, blocos com erro não impedem o acesso aos demais blocos do arquivo.	Baixa: erro em um bloco leva à perda dos dados daquele bloco e de todos os blocos subsequentes do arquivo.	Alta, desde que não ocorram erros na tabela de alocação.	Alta, desde que não ocorram erros no *i-node* nem nos blocos de ponteiros.	Alta, blocos com erro não impedem o acesso aos demais blocos do arquivo.
Flexibilidade	Baixa, o tamanho máximo dos arquivos deve ser conhecido a priori; nem sempre é possível aumentar o tamanho de um arquivo existente.	Alta, arquivos podem ser criados em qualquer local do disco, sem risco de fragmentação externa.	Alta, arquivos podem ser criados em qualquer local do disco, sem risco de fragmentação externa.	Alta, arquivos podem ser criados em qualquer local do disco, sem risco de fragmentação externa.	Alta, mas há risco de fragmentação excessiva e queda de desempenho.
Limites	O tamanho de um arquivo é limitado ao tamanho do disco.	O número de bits do ponteiro limita o número de blocos endereçáveis e o tamanho do arquivo.	O número de bits do ponteiro limita o número de blocos endereçáveis e o tamanho do arquivo.	Número de ponteiros no *i-node* limita o tamanho do arquivo; tamanho da tabela de *i-nodes* limita número de arquivos.	O tamanho de um arquivo é limitado ao tamanho do disco.

Gestão do espaço livre

- **Mapa de bits:** 1 bit por bloco; simples e compacto.
- **Lista de blocos livres:** cada bloco livre aponta para o próximo.
- **Tabela de extensões livres:** [bloco inicial, tamanho].
- Na FAT, os blocos livres são marcados na própria tabela.

Falhas e recuperação

Falhas por crash

- Criar um arquivo exige várias escritas: mapa de inodes, inode e diretório.
- Um crash no meio deixa o sistema de arquivos **inconsistente**.
- Perder metadados pode perder subárvores inteiras de diretórios.

Verificação e journaling

- **fsck** percorre todas as estruturas e pode levar horas.
- **Journal:** as alterações são registradas antes de serem feitas.
- Após um crash, o SO completa as operações pendentes.
- Em geral, o journal cobre só os metadados.

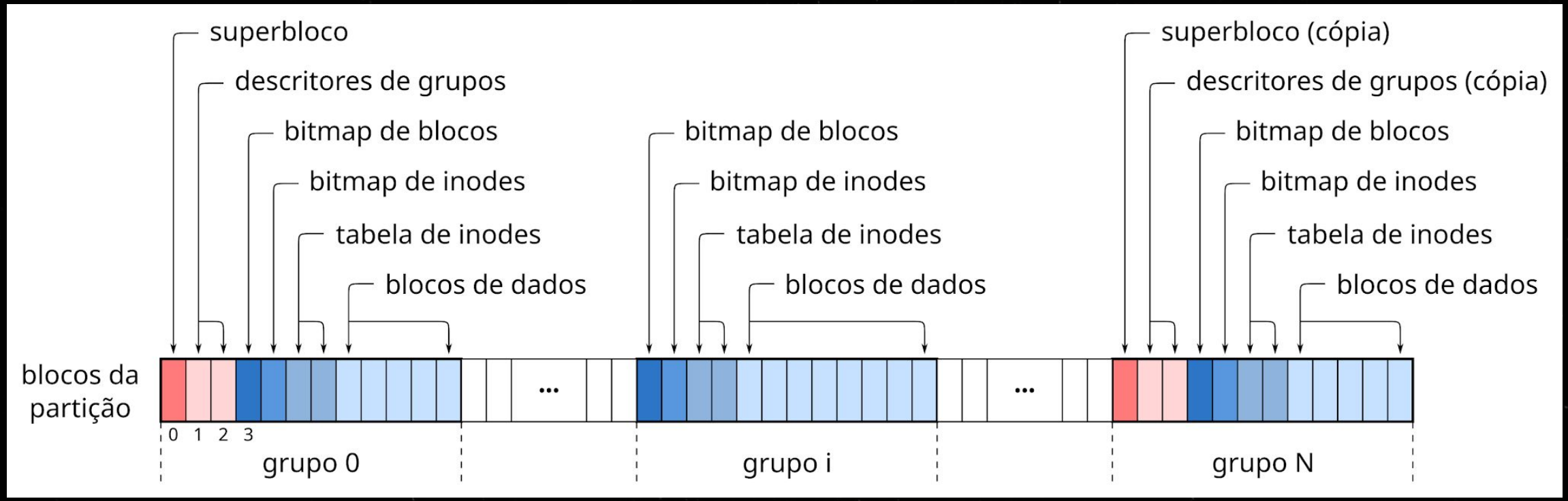
Copy-on-write

- O novo conteúdo vai para um **bloco livre**; o antigo é preservado.
- Mantém **snapshots** de estados anteriores.
- Após um crash, volta ao último snapshot consistente.
- Usado em ZFS e BTRFS.

Exemplo: Ext4

- Sistema de arquivos mais usado no Linux.
- Journaling e checksum de metadados.
- Alocação indexada multinível e por extensões.
- Espaço livre controlado por bitmaps de blocos e de inodes.

Estrutura do Ext4



Resumo

- Dados e metadados
- Blocos e caching
- Estratégias de alocação
- Gestão do espaço livre
- Consistência após falhas

Atividade

1 - Explique as diferentes formas de alocação de memória, apresentando suas vantagens e desvantagens.

2 - O que é fragmentação interna e fragmentação externa? Por que elas ocorrem?

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

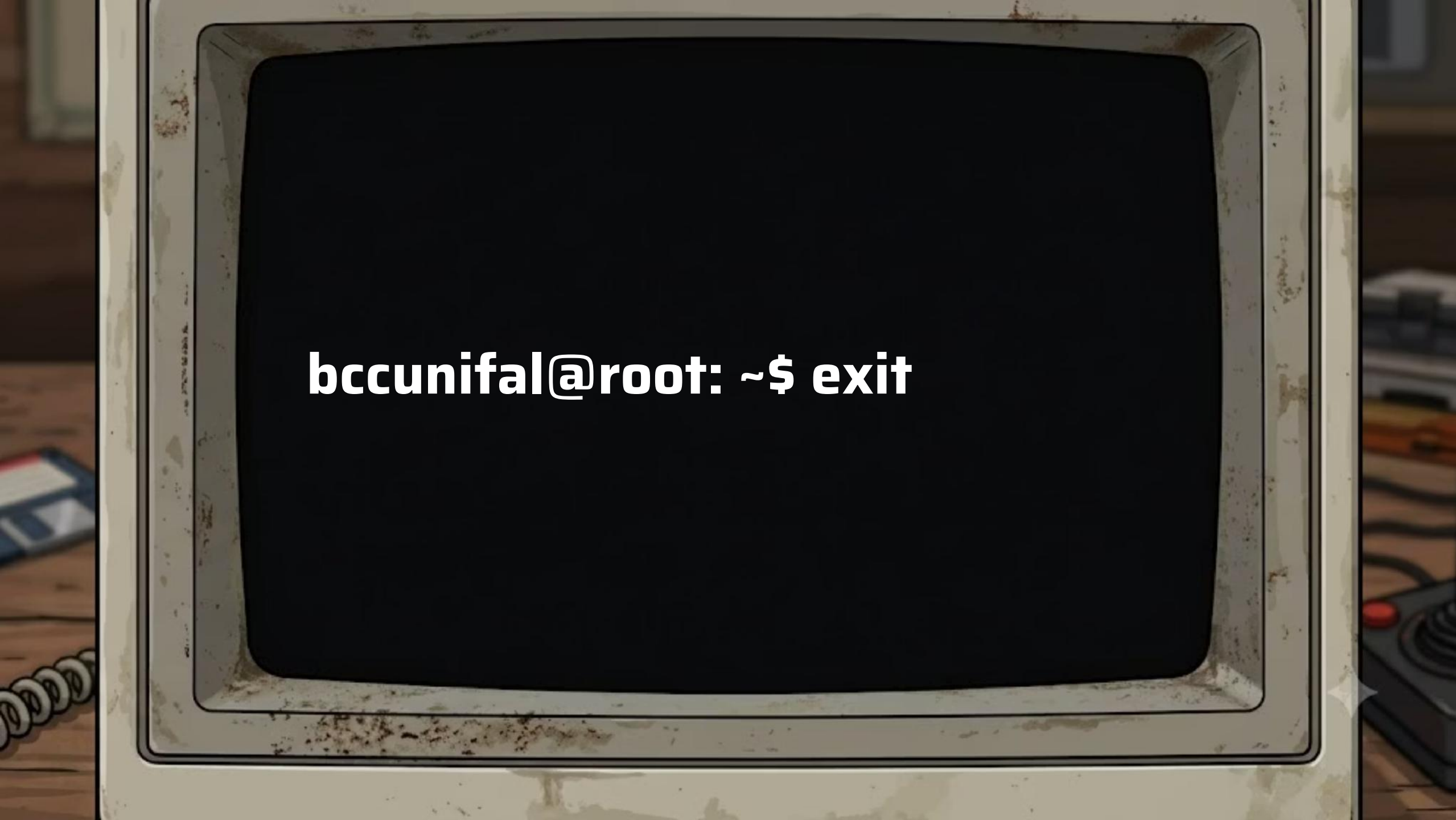
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.


bccunifal@root: ~$ exit